HaskLedger - Project Closeout Report

This report tells the full story of HaskLedger for the Cardano community: what was built across five Catalyst milestones, how it performs, how it was validated, what changed along the way, and where it goes next. The formal milestone evidence is in the Milestone 5 Proof of Achievement.

1. Project overview

HaskLedger is a type-safe Haskell eDSL for Cardano smart contracts, funded by Project Catalyst (Fund 11) and built by Konma. Contracts are written in ordinary Haskell - `do`-notation, operator overloading (`.==`, `.&&`), integer literals - and compiled to Plutus V3 UPLC through MLabs' Covenant intermediate representation (v1.3.0) and c2uplc code generator (v1.0.0), emitting standard `.plutus` envelopes deployable with `cardano-cli`.

Five milestones: (1) Cardano ecosystem analysis, (2) design document and validation, (3) core functionality development, (4) prototype development and internal testing, (5) community engagement, feedback integration and closeout.

Repository: https://github.com/KonmaORG/HaskLedger (Apache-2.0). Contact: techteam@konma.io.

Item	Value
Catalyst project ID	1100154
Challenge	F11: Cardano Open: Developers
Budget	ADA 200,000
Funds distributed (at Milestone 5 submission)	ADA 170,000
Start date	March 11, 2024
Change requests	August 27, 2024; August 27, 2025
Delivery	September 2026

2. Original problem and proposed solution

The Milestone 1 ecosystem analysis identified three gaps that framed the project:

1. **Developer onboarding.** Plutus development demands deep Haskell and platform knowledge at once; the learning curve keeps working developers out of the ecosystem, shrinking the pool of auditable contracts.
2. **Smart contract cost and performance.** Scripts produced by the standard toolchain are large and execution-unit-hungry, which shows up as fees, block-space pressure, and applications that are not viable on-chain.
3. **Hardware breadth.** Limited support for building and validating the toolchain beyond x86_64, including open-source hardware (RISC-V).

The proposed answer, refined in the Milestone 2 design document after MLabs' design review (section 7): a typed Haskell eDSL over a restricted, deterministic validator subset, compiled through a purpose-built IR to lean Plutus Core - Haskell ergonomics without PlutusTx's output cost profile. The design document tabulated the allowed eDSL subset (total pattern matches, structural folds, no unbounded recursion, no effects), positioned HaskLedger against PlutusTx, Plutarch, Aiken, and Helios, and set the testing bar (golden, round-trip, and differential tests). Section 6 shows the central bet paying off, measured.

3. Architecture

```
HaskLedger eDSL -> Covenant ASG -> Covenant JSON -> c2uplc -> UPLC -> .plutus
envelope
```

The combinator library. Users write against a `Contract` monad: `validator`, `require`/`requireAll`, ledger accessors (`theRedeemer`, `txValidRange`, TxInfo field access), comparison and boolean operators, Data destructuring (`asInt`, `asMap`, constructor field access), crypto (blake2b, sha2, keccak, signature verification), and temporal constraints (`after` alone hides 10+ levels of interval/bound/closure-flag destructuring). One `import HaskLedger` brings in everything.

The IR. Covenant represents programs as a hash-consed abstract syntax graph: structurally identical subexpressions share one node, so two combinators that both destructure the ScriptContext pay for that destructuring once. c2uplc performs lambda lifting, type erasure, and de Bruijn conversion to produce the final UPLC.

Field-selective destructuring. The pipeline only decodes the ScriptContext fields a contract actually reads. This is the structural reason for the benchmark gap in section 6: idiomatic PlutusTx decodes the entire context up front (~25.5M-step floor on every contract), HaskLedger does not.

Builtin-only compilation policy. The eDSL deliberately compiles through raw Plutus builtins (`builtin1/2/3/6` + `lam`/`app'`/`thunk`/`force`/`lit`) and bypasses Covenant's higher-level `match`/`ctor'` primitives, because c2uplc's Transform pipeline has five documented defects that corrupt UPLC output for those forms (strictness, duplicate projections on hash-consed nodes, let-nesting order, handler scope confusion, name-counter collision).

Error model. Validators succeed by returning unit and fail via an unrecoverable runtime error: `require` compiles to a guarded `DivideInteger 1 (IfThenElse cond 1 0)` - division by zero on a false condition. Branch expressions are thunked (`delay`/`force`) where needed, because UPLC application is strict.

Depth-tracked expressions. Closures that capture an outer lambda's argument need de Bruijn shifting at every capture site. HaskLedger's `Expr` is a depth-tracked recipe re-derived at its use depth, which makes capture correct uniformly across nested folds and branches (see section 9 for the bug class this closed).

4. Work completed per milestone

Milestone	Delivered
M1 - Ecosystem analysis	Cardano ecosystem gap analysis: SWOT, technical analysis of onboarding, script cost/performance, hardware compatibility, EUTxO concurrency; identified the opportunity for a typed eDSL with leaner output
M2 - Design & validation	Architecture and design document, rewritten after MLabs' design review: eDSL -> Plutus Core compilation pipeline, tabulated deterministic validator subset, competitive positioning (PlutusTx / Plutarch / Aiken / Helios / Covenant-style), testing strategy (golden / round-trip / differential), build-profile plan incl. riscv64 via QEMU; expert consultations recorded
M3 - Core functionality	Working toolchain: eDSL -> Covenant IR -> c2uplc -> UPLC -> `.plutus`; contract-construction and combinator API; first 4 contracts (always-succeeds, redeemer-match, deadline, guarded-deadline) compiled, deployed, and spent on Preview testnet (cardano-node 10.5.4); docs + test suite
M4 - Prototype & internal testing	9 contracts validated on-chain with positive and negative cases each (cardano-node 11.0.1, Plutus V3); minting-policy support (one-shot NFT); reproducible deploy scripts with raw logs in `deploy-out/`; offline throughput/efficiency benchmark vs idiomatic PlutusTx (initially not approved; approved after the benchmark addendum - see section 9)
M5 - Community engagement & closeout	Final 13-contract set with security hardening; three external reviews of the finished prototype; internal developer review; documented expert validation and outreach; a response and outcome for every recorded feedback item; this report; closeout video

Milestone PoA documents are in the repository root (section 14).

5. Technical capabilities delivered

- Spending validators and minting policies from one `import HaskLedger`
- 13 contracts, all deployed and validated on the Cardano Preview testnet with positive and negative cases: always-succeeds, redeemer-match, deadline, guarded-deadline, hash-lock, hash-verify, oracle, treasury, one-shot-nft, escrow, vesting, token-gate, multisig
- Datum-driven configuration: parties, deadlines, signature thresholds, and gate tokens are datum values - a 3-of-5 multisig instead of 2-of-3 is a datum change, not a new contract
- Security hardening pass over all 13 contracts: datum hijack, token-name smuggling, double satisfaction, and dust payout closed by construction, with dedicated guard combinators (`paysAtLeast`, `singleOwnScriptInput`, `inlineDatumEquals`, `ownMintTokenCount`); threat-model headers state what each contract does and does not guarantee (docs/advanced-contracts.md, docs/contract-hardening.md)
- 7 test suites, 420 tests green; offline benchmark harness (`haskledger-bench`) with a committed idiomatic-PlutusTx baseline
- Advanced contracts compile small: escrow 2,478 bytes, vesting 1,468, token-gate 944, multisig 518 - each from roughly twenty lines of source
- Cross-platform: x86_64/aarch64 Linux + macOS; riscv64-linux validated end-to-end via GHC 9.12.2's native RISC-V code generator under QEMU

6. Benchmark and efficiency results

Same logic, byte-identical ScriptContext inputs, measured with the chain's cost model (plutus-core 1.51 default CEK parameters) against idiomatic PlutusTx compiled with the official plugin at the same plutus version. Ratios are PlutusTx over HaskLedger.

Contract	HL bytes	PlutusTx bytes	size ratio	HL steps	PlutusTx steps	steps ratio	HL mem	PlutusTx mem	mem ratio
always-succeeds	161	2,533	15.7x	976,100	25,561,498	26.2x	6,200	101,575	16.4x
redeemer-match	192	2,545	13.3x	1,984,619	25,794,575	13.0x	9,662	102,608	10.6x
deadline	372	3,258	8.8x	10,710,190	30,778,058	2.9x	32,383	132,941	4.1x
guarded-deadline	407	3,269	8.0x	11,860,171	31,118,972	2.6x	36,349	134,375	3.7x
hash-lock	223	2,561	11.5x	3,833,672	26,528,932	6.9x	14,082	105,077	7.5x

Per-block script capacity under Cardano's block execution budget (62,000,000 memory units, 20,000,000,000 steps):

Contract	HaskLedger scripts/block	PlutusTx scripts/block
always-succeeds	10,000	610
redeemer-match	6,416	604
deadline	1,867	466
guarded-deadline	1,686	461
hash-lock	4,402	590

Headline: 8-16x smaller scripts, 2.6-26x fewer CPU steps, 3.7-16x less memory. The structural cause is field-selective destructuring - idiomatic PlutusTx pays a ~25.5M-step floor decoding the whole ScriptContext on every execution. The heaviest contract (treasury withdraw) uses 1.5% of one transaction's memory budget and 0.7% of its step budget. Everything reproduces offline with `cabal run haskledger-bench`; the harness exits nonzero if any validator rejects its positive case, so published numbers cannot come from a silently failing script.

Full tables (all contracts, fees, capacity bounds): `haskledger/bench/bench-results.md`. Methodology and stated limits (idiomatic baseline, derived capacity, minimal contexts are the conservative case): `F-11-Milestone-4-Throughput-Efficiency-Addendum.md`.

7. Community testing and validation

HaskLedger sought outside scrutiny from its design stage onward, not only at closeout.

External review of the final prototype. On 23 September 2026, Koz Ross, head developer of Covenant at MLabs, reviewed HaskLedger's Haskell implementation and documentation (public repository at commit `84078da`). He found HaskLedger "usable and useful today", with very strong performance, an eDSL that is easy to follow, and a separate front end and back end that can improve independently. His concerns were sparse user-facing documentation, too little explanation of the compilation strategy and outputs, and the lack of testing helpers comparable to Plutarch and Aiken, with Aiken-level documentation and tooling as the long-term target. The review and the response to each point are published in the review record. The review is also available as an official letter on MLabs letterhead: koz-ross-mlabs-2026-09-23-letter.pdf.

Two more external reviews of the final prototype. On 25 and 26 September 2026 the prototype was reviewed by Suganya Raju (Cardano tooling and open-source developer experience) and Harun Mwangi (Cardano smart-contract developer and architect). Suganya Raju found it ready for wider technical evaluation, with a clean front-end/back-end split, reproducible claims and convincing negative tests, and asked for user documentation as its own entry point, a compatibility table, CI for contributors and a stated line between stable and internal APIs. Harun Mwangi found no technical issue that would stop it being used today as an experimental to early-production framework, and asked for better onboarding, error reporting that does not require reading UPLC, benchmark claims kept to the measured contracts, and adoption by independent developers. The reviews and responses are published: Suganya Raju, Harun Mwangi, and both reviewers confirmed them on GitHub (PR #14, PR #16). Every finding from all three reviews is a public GitHub issue with the `review` label, and the reviews are also posted on the pinned review issue.

Internal developer review. On 19 July 2026, Vinit Inamke (HaskLedger core developer, from the Fund 7 Konma Labz Haskell cohort) and Sangeet Muralidhar (Head of Technology, Konma) reviewed the prototype at commit `14999b8`, covering setup from the project documentation, the example contracts and benchmark harness, and deliberate failure cases with invalid datum and redeemer inputs.

Design-stage expert validation (2024). Before the prototype existed, feedback on the design was gathered from engineers and researchers across the Haskell and Cardano ecosystem. This is historical context and was assessed under Milestone 2:

Expert	Background	Engagement
Duncan Coutts	Well-Typed; IOHK	Consultation meeting, September 2024 (not recorded, at his request; key takeaways documented)
Koz Ross	Software Development Lead, MLabs	Written and recorded review of the Milestone 2 design document
Björn Kihlberg	Former Haskell and Marlowe developer, IOHK	Written feedback on the technical stack, August 2024
Adithya Obilisetty	Haskell engineer, Composewell	Written feedback on the technical stack, August 2024
Claudio Hermida	Honorary Research Fellow, University of Birmingham	Written feedback on scope and formal semantics, October 2024

Expert	Background	Engagement
Kapil Shyam	System Software Engineer, Mindgrove Technologies (Shakti RISC-V)	Written feedback on RISC-V and security, October 2024
Intersect MBO	Cardano member-based organisation	Recorded sessions with feedback on data management and education

Outreach. Between August 2024 and February 2025 the team also contacted more than 30 further researchers and engineers on LinkedIn and X, sharing the design document and a structured feedback form. Most did not respond or declined, citing existing commitments.

The originally planned external review. The proposal anticipated review of the prototype by developers or consultants associated with Well-Typed, Tweag and Hasura. Beyond the design-stage consultation with Duncan Coutts, a completed, attributable review of the finished prototype from those organisations could not be secured. This report does not represent any such review as having occurred.

Technical engagement with MLabs. Building a full contract library on MLabs' Covenant and c2uplc toolchain produced sustained technical engagement with it: five Transform-pipeline defects documented and worked around with the builtin-only policy, code-generation scoping defects fixed with local patches to the vendored c2uplc, and a `cata` handler argument-order mismatch between Covenant's typechecker and c2uplc's codegen reported to the Covenant maintainers with a minimal reproduction.

The expert feedback, meeting takeaways and outreach evidence are published in the Community and Expert Validation and MLabs Validation records.

8. Feedback-driven improvements

Every recorded feedback item has a documented response and outcome in the Milestone 5 Proof of Achievement.

From the prototype review (EXT-ML-01 to EXT-ML-04). The documentation was rewritten for library users in response to Koz Ross's review, in commit `7141aa6`:

- **Sparse user documentation (implemented).** A documentation index, getting started, an API reference covering every exported function, a task-based user guide, the datum and redeemer of every example contract, and a security guide.
- **Compilation strategy and outputs (implemented).** A compilation page covering the pipeline stages, the Plutus V3 types as Plutus Data, laziness, sharing, `cata` loops, variable naming and how to inspect each stage's output.
- **Testing helpers (partially implemented).** A testing guide documents the off-chain helpers with a worked example. Shipping them as a library module is still to do.
- **Aiken-level documentation and tooling (long-term target).** A comparison page states the current gaps: typed datums, a built-in test runner and CIP-57 blueprints.

From the two further prototype reviews (EXT-SR, EXT-HM; 8 items). 4 implemented, 1 partially implemented, 2 accepted for the roadmap, 1 ongoing:

- **Compatibility (implemented).** A compatibility page lists the tested GHC, Covenant, c2uplc and Plutus versions, the local c2uplc fixes, and each known upstream issue with its workaround.
- **API stability (implemented).** The API reference now says what is stable: what `import HaskLedger` exports; the internal modules can change.
- **Onboarding (implemented / partially implemented).** Getting started now has a troubleshooting table and says which example to start from. A template generator is not built.
- **Scoped benchmark claims (implemented).** Already stated as results for five contracts.
- **Roadmap.** CI for contributors, and failure reporting without reading UPLC. The README had wrongly listed CI on its platform table; that is corrected.
- **Ongoing.** Independent developers building their own contracts, invited through the pinned review issue.

From the design stage. 17 items, of which 8 were implemented, 5 partially implemented, and 4 not implemented with a technical rationale.

Implemented.

- **A technical, specific design.** MLabs found the first design document too broad and marketing-styled. It was rewritten as a technical specification, and the funded milestones were delivered as a concrete, testable toolchain.
- **A simpler, mainstream stack.** Experts warned the early concept (RISC-V, NuttX, unikernels, Haskell, Rust, Racket, Guix) was too diverse. The delivered toolchain is Haskell end to end, the DSL is embedded in Haskell rather than Racket, and the build is standardised on Nix flakes rather than Guix.
- **Measured, not asserted, performance.** MLabs asked for performance improvements stated against a target. Performance is now evidenced only by the reproducible PlutusTx benchmark in section 6.
- **Cardano compatibility.** HaskLedger compiles to standard Plutus V3 scripts validated on unmodified Cardano, the no-fork path Duncan Coutts described.
- **Assessable scope.** Claudio Hermida found the project very ambitious; the design separates a short-term track from long-term work, and each milestone shipped with verifiable evidence.

Partially implemented. RISC-V support sits at the toolchain level (GHC 9.12.2's native RISC-V backend, validated under QEMU) but not yet on physical hardware. The design keeps heavy computation off-chain with Cardano for settlement, as Intersect MBO suggested, but only the on-chain layer was funded. Documentation is extensive, but a broader education campaign has not been run.

Not implemented, with rationale. Running Embedano on bare-metal RISC-V, unikernel deployment, formal semantics for custom instruction sets, and Rust for security-critical components all belong to the embedded and off-chain tiers of the early concept, outside the on-chain toolchain the milestones delivered. On-chain security is instead enforced in the contracts themselves through the audited guard combinators.

9. Deviations, challenges, and lessons

Scope focus vs the M2 design. The design document described both the on-chain compilation pipeline and a longer-term off-chain runtime (actor pipelines, gateway, sidechain checkpointing). The funded milestones delivered the on-chain toolchain end to end; the off-chain runtime remains documented architecture on the roadmap (section 13). RISC-V validation stayed in scope via the QEMU smoke test the design planned, not physical boards.

The Milestone 2 design review. MLabs' review of the first design document was critical: too broad, light on specifics, performance claims unjustified. The document was rewritten, and that review shaped the rest of the project - a concrete toolchain, stated roles for each language, and performance proven by measurement.

M4 rejection and resubmission. M4 was initially not approved (July 24, 2026) for not evidencing "improvements in transaction throughput and efficiency" - the original PoA reported on-chain fees but defined no baseline. The response was the offline benchmark harness plus a committed idiomatic-PlutusTx baseline compiled with the official plugin on byte-identical inputs, with limitations stated up front. Approved on resubmission, August 2026.

The external review pathway. The proposal named Well-Typed, Tweag and Hasura as external reviewers. The Haskell and Cardano ecosystem, and the availability of those independent organisations, changed materially between Fund 11 and closeout. Despite a design-stage consultation with Well-Typed's Duncan Coutts, direct feedback from other experts, and outreach to more than 30 more, a completed review of the finished prototype from the three named organisations could not be secured. The finished prototype was instead reviewed by three external engineers: Koz Ross (MLabs), Suganya Raju and Harun Mwangi. This is documented openly rather than papered over.

Covenant/c2uplc Transform pipeline defects. Five distinct defects in c2uplc's Transform pipeline corrupt UPLC output for Covenant's `match`/`ctor'`/`lazyLam` forms (strict ChooseData branches, duplicate projections on hash-consed nodes, wrong let-nesting order, scope confusion across handler boundaries, name-counter collision). Rather than block, the project documented them and adopted the builtin-only compilation policy. A further mismatch (cata handler argument order between Covenant's typechecker and c2uplc's codegen) was found during the hardening pass and reported to the Covenant maintainers with a minimal reproduction.

The de Bruijn capture bug class. The hardest bug of the project: reusing expression references across lambda boundaries without de Bruijn shifting produced ill-formed ASGs, which hash-consing then conflated with structurally identical local expressions - the information needed to fix it downstream was destroyed at construction time. Root-caused in July 2026 and closed by making `Expr` a depth-tracked recipe re-derived at each use depth; 19 remaining contract failures went to zero, 420/420 tests green.

Timeline. The project started March 11, 2024, with two change requests filed along the way (August 27, 2024 and August 27, 2025). Original closeout was planned for February 2026; actual delivery moved to September 2026. The main sinks were the capture bug class (weeks of diagnosis against a vendored compiler), the M4 resubmission cycle, the full security-hardening pass (four audit findings closed), and the long external-review outreach.

Lessons. Measure against a baseline from day one. Write design documents as specifications, not pitches. State limitations in the source, not the postmortem. When a compiler misbehaves, dump and read the actual UPLC before forming hypotheses. On-chain negative tests - the validator itself refusing a bad transaction at build time - are stronger evidence than any unit test.

10. Open-source outputs

Output	Link
Repository (Apache-2.0)	https://github.com/KonmaORG/HaskLedger

Output	Link
Contracts, tests and deploy scripts	`haskledger/examples/`, `haskledger/test/`, `haskledger/deploy/`
On-chain deployment logs	`deploy-out/`
Benchmark harness + results + PlutusTx baseline	`haskledger/bench/`
Documentation	README, user guide, architecture, deployment guide, advanced contracts, contract hardening, Haddock API docs
Capture-bug fix design	`docs/option-a-depth-tracked-expr.md`
Vendored MLabs toolchain, with local c2uplc fixes	covenant/, c2uplc/
Milestone PoAs incl. throughput addendum	`F-11-Milestone-*.md` in the repository root (section 14)

11. Benefits to the Cardano ecosystem

Working Haskell developers get an on-ramp to Cardano without PlutusTx's cost profile: ordinary Haskell in, scripts 8-16x smaller out, which translates directly to lower fees and more validating transactions per block on unmodified Cardano. The benchmark harness gives the ecosystem a reproducible method for comparing contract toolchains - committed baseline, chain cost model, offline reproduction - that any other toolchain can drop its own envelopes into. The documented Covenant/c2uplc defects and the reported `cata` mismatch help everyone building on MLabs' IR. The security-hardening pass documents four attack classes (datum hijack, token smuggling, double satisfaction, dust payout) with by-construction defenses that other eDSL authors can copy.

HaskLedger is also in real-world use as the underlying smart-contract technology for Karbon Ledger, Konma's climate-tech product for emissions and compliance tracking. UNDP features Karbon Ledger in *New Tech, New Partners: Transforming Development in the Digital Era*, its snapshot of blockchain practice from the UNDP Alternative Finance Lab, as the solution maker for a UNDP India pilot: Streamline, which connects IoT sensors, analytics and a blockchain-based compliance log to give Common Effluent Treatment Plant operators and regulators real-time visibility, starting with the plastic recycling and textile clusters.

12. Current limitations

Stated in the source and docs, not discovered by users:

- **Payout floors are lovelace-only.** `paysAtLeast` and `valuePreserved` measure ADA; native-token amounts are not part of the payout check (escrow, vesting).
- **No short-circuit evaluation.** UPLC application is strict: `.&&`/`.||` evaluate both sides, `matchBool` evaluates both branches. A predicate that crashes on one list element crashes the whole iteration.
- **Token-gate checks presence, not quantity.** It is an access check, not a payment rule - by design, and documented.
- **Multisig keys must be distinct.** Cardano deduplicates a transaction's required signers, so a duplicated datum key still counts once (ledger-level, documented).

- **Hash-lock preimages are mempool-visible.** Classic front-running exposure inherent to the pattern.
- **Builtin-only policy.** Covenant's `match`/`ctor'` forms are not used, because of the Transform-pipeline defects in c2uplc; the eDSL's own pattern-matching combinators cover the gap.
- **Script purposes.** Spending validators and minting policies; staking/governance purposes are not yet exposed.
- **RISC-V on emulation only.** riscv64 is validated under QEMU, not yet on physical hardware.
- **Benchmark baseline covers 5 of the contracts** (the five lowest complexity tiers); the rest are measured on the HaskLedger side in full.
- **No CI yet.** Changes are checked with the documented build, test, benchmark and example commands.

13. Future roadmap

- **Datum-parametric public beta.** Upgrade the remaining fixed-configuration contracts to full inline-datum parametrization (the pattern escrow/vesting/token-gate/multisig already use) for a public beta release.
- **Native-token payout floors** and richer Value checks, closing the lovelace-only limitation.
- **Physical RISC-V validation** (e.g. StarFive VisionFive 2), extending the QEMU result to hardware.
- **Off-chain runtime** from the M2 design (actor pipelines, gateway, checkpointing) as a separate, future-funded track, including the off-chain storage with on-chain verification Intersect MBO recommended.
- **Education resources** - tutorials and learning material beyond the reference documentation, as Intersect MBO recommended.
- **Developer tooling from the prototype reviews** - CI for contributors, reporting which check failed without reading UPLC, the test helpers as a library module, and PlutusTx baselines for escrow, vesting, multisig and treasury.

14. Links

Item	Link
Repository	https://github.com/KonmaORG/HaskLedger
Release	`v1.0.0`
External prototype review, Koz Ross (MLabs)	review record and feedback log
External prototype reviews, Suganya Raju and Harun Mwangi	Suganya Raju, Harun Mwangi
Review findings as GitHub issues	`review` label
Closeout video (public, YouTube)	https://www.youtube.com/watch?v=rUCvvvjgJSc

Item	Link
Closeout video (archival original, Google Drive)	https://drive.google.com/file/d/1UBPChPF602Y4wYDq03usrs1qU4d96eSj/view?usp=sharing
Milestone 5 PoA	`F-11-Milestone-5-Community-Engagement-Feedback-Integration-Closeout.md`
Milestone 4 PoA and throughput addendum	`F-11-Milestone-4-Prototype-Development-and-Internal-Testing.md`, `F-11-Milestone-4-Throughput-Efficiency-Addendum.md`
Milestone 3 PoA	`F-11-Milestone-3-POA-Core-Functionality-Development.md`
Milestone 2 PoA	`F-11-Milestone-2-POA-Design-Document-Validation.md`
Milestone 1 PoA	`F-11-Milestone-1-POA-Cardano-ecosystem-analysis.md`
Community and Expert Validation	https://konmadao.notion.site/Community-and-Expert-Validation-for-HaskLedger-6e5fd20d028847618fd44a4d50b1f5e3
MLabs Validation	https://konmadao.notion.site/MLabs-Validation-28d468b438dc80d4be83e4cd2ab02ea0
UNDP publication featuring Karbon Ledger	`docs/undp-new-tech-new-partners-transforming-development-in-the-digital-era.pdf` (page 57)
M3 demo video	https://drive.google.com/file/d/1J12H415zmsEjZNNFyXoWDJ7tMknttHSD/view
M4 demo video	https://drive.google.com/file/d/1YxsfksK8i5yn1sVuGS-GSZX6RDj60Hz1/view